

COMP2054-ADE

Algorithms Data Structures & Efficiency

ADE Lec01

Analysis of Algorithms

Lecturer: Andrew Parkes

Email: andrew.parkes@nottingham.ac.uk

<http://www.cs.nott.ac.uk/~pszajp/>

Running Time: “finite” but how big?

- Consider “batch algorithms”: transform input data into output data – as opposed to “interactive”
- The running time of an algorithm typically grows with the input size.
- Even at given size the runtime is usually not fixed.
 - We can have “best”, “average” and “worst” cases.
- We (usually) focus on the worst-case running time at given size, since
 - Worst-case is usually most useful and easiest (not “easy”!) to analyse
 - Average case time is often difficult to determine. Worse yet: “average” can refer to “mean” or “median” or other measures, and these might behave quite differently
 - Best-case rarely matters (?)

Experimental Studies

General Pattern:

- Write a program implementing the algorithm
- Run the program with inputs of “varying size and composition”
- Use a system method to get an (in)accurate measure of the actual running time
- Interpret & analyse the results.

E.g. is it

- A “power law”, n^k , for some k
- An “exponential”, b^n , for some b

Running Time: how big?

Suppose you have an algorithm and some experiment gives running times (in some time units, such as 'ms')

Size n	Worst	Average	Best
100	10.3	9.5	2.2
200	19.7	14.1	2.3
300	29.6	17.5	2.1
400	40.3	19.6	2.2
500	50.2	22.8	2.4

We need methods to be able to answer questions such as

- How would you interpret / analyse these results?
- How would you communicate to others the scaling of the algorithm? I.e. how much does it slow down for larger instances?

Limitations of Experiments

- It is necessary to implement the algorithm, which may be difficult or time-consuming
- **Results may not be indicative of the running time on other inputs not included in the experiment.**
 - **Maybe we miss the “real worst case”**
- To compare two algorithms directly, the same hardware and software environments must be used

Limitations of Theory

- It is necessary to implement the theory, which may be difficult or time-consuming
- Results may not be indicative of the typical running time on inputs encountered in real world.

So can be useful to be able to use both experiment and theory.

Aside: Theory vs. Experiment

Standard science:

**“Never believe a theory until it has been
‘confirmed’ by an experiment”**

Partially joking:

**“Never believe an experiment until it has
been confirmed by a theory”**

- **Attributed to Sir Arthur Eddington**
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3597502/>

Theoretical Analysis

- **AIM: Characterise running time as a function of the input size, n .**
- Use a “high-level” description of the algorithm instead of an implementation, so that we
 - Take into account all possible inputs
 - Can evaluate the speed of an algorithm independently of the hardware/ software/ language environment

Pseudocode (recap)

- High-level description of an algorithm
 - More structured than English prose
 - Less detailed than a program
 - Preferred notation for describing algorithms
 - Hides program design issues
-
- Will use a C/Java style of syntax, but usually ignore “{ }” or “;” and use a Python style of “indentation for grouping”:

Pseudo-code Example

```
// get max of a non-empty array
int getMax(int[] A) { // A has size n
    max = A[0]
    for (int k = 1; k < n ; k++ )
        if ( A[k] > max )
            max=A[k]
    return max
}
```

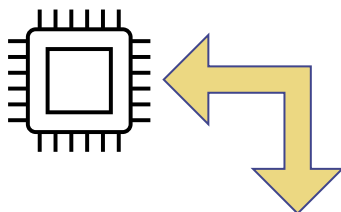
To analyse programs we will count “Primitive Operations”:

“Primitive Operations”

- Basic computations performed by an algorithm
 - Identifiable in pseudocode
 - Largely independent from the programming language
 - Exact definition not important (we will see why later)
- Assumed to take a constant amount of time in the “RAM model” (next slide)
 - Tend to be close to “Assembly language”
 - No “hidden expenses”

The “Random Access Machine” (RAM)

- A **CPU**
- A potentially-unbounded bank of **memory** cells, each of which can hold an arbitrary number or character



0: 3	1: 1	2: 4	3: 1	4: 5	5: 9	...
-------------	-------------	-------------	-------------	-------------	-------------	------------

- Memory cells are numbered 0,1,... and accessing any cell in memory takes unit time (some fixed time).
 - Note that RAM can stand for both “Random Access Machine” and “Random Access Memory,” Which is an unfortunate, but standard, over-loading of terminology.

Limitations of RAM model

- “... can hold an arbitrary number ...” ?
 - Can we really expect to store
“938566359286151800351666177735777777177177374717717471
5717777761365661618161616” in one cell on a real computer?
- Here, we ignore such “bignum” issues. Instead:
- “all numbers are of equal size, as they all fit in a single register of the CPU”
- 64bits (signed int) allows up to 9,223,372,036,854,775,807
 - Exercise (offline): compare to: nanoseconds since big bang;
national debt.
- Note: on real machines (usually) computing
1 + 1 takes as long as 381513 + 243542
Hence, we typically ignore the sizes of numbers in the arithmetic operations.

“Primitive Operations” examples

- Assign value to a variable
- Compare two numbers
- Perform one arithmetic operation – add, subtract, multiply, divide, modulus, etc
- Call a method
- Return a value

GetMax: Counting (1)

We will count the maximum number of Prim.Ops.
associated with each line of the pseudo-code

Num.Prim.Ops

// get max of a non-empty array

int getMax(int[] A) { // A has size n

 max = A[0] ?

 for (int k = 1; k < n ; k++) ?

 if (A[k] > max) ?

 max=A[k] ?

 return max ?

TOTAL OPS? ?

If offline: pause and try it yourself on all slides like this!

GetMax: Counting (2)

Partial start. Will split by cost per line, and multiply by number of passes.

	Num.Prim.Ops		
	Per.Line	Num.Pass	
int getMax(int[] A) { // A has size n			
max = A[0]		?	
for (int k = 1; k < n ; k++)		?	
if (A[k] > max)		?	
max=A[k]	2	*	(n-1)
return max		?	
TOTAL OPS?			?

count used here: 1 for A[k] lookup, 1 for assignment

GetMax: Counting (3)

Partial start. Will split by cost per line, and multiply by number of passes.

	Num.Prim.Ops		
	Per.Line	Num.Pass	
int getMax(int[] A) { // A has size n			
max = A[0]		2	
for (int k = 1; k < n ; k++)		?	
if (A[k] > max)	2	*	(n-1)
max=A[k]	2	*	(n-1)
return max		1	
 TOTAL OPS?			?

count used here: 1 for A[k] lookup, 1 for ">" test

GetMax: Counting

(4)

Split up the “hidden costs” of the “for” loop

	Num.Prim.Ops	
	Per.Line	Num.Pass

int getMax(int[] A) { // A has size n		
max = A[0]		2
for (int k = 1; k < n ; k++)		1 {for k=1}
if (A[k] > max)	2	* (n-1)
max=A[k]	2	* (n-1)
{increment k}	?	
{test k < n}	?	
return max		1
TOTAL OPS?		?

GetMax: Counting

(5)

Split up the “hidden costs” of the “for” loop

	Num.Prim.Ops		
	Per.Line	Num.Pass	
int getMax(int[] A) { // A has size n			
max = A[0]		2	
for (int k = 1; k < n ; k++)		1	{for k=1}
if (A[k] > max)	2	*	(n-1)
max=A[k]	2	*	(n-1)
{increment k}	1	*	(n-1)
{test k < n}	1	*	(n-1)
return max		1	

TOTAL OPS?

$6n - 2$

Counting is “underspecified” (1)

- Consider “ $c = A[i]$ ” then ‘full’ process can be
 - get A = pointer to start of array A , and store into a register
 - get i , and store into a register
 - compute $A+i$ which is a pointer to location of $A[i]$, and store back into a register
 - get value of “ $*(A+i)$ ” (from RAM) and store value it into a register
 - copy the value into the location of c in the RAM
- might not want to count all this, e.g. just count
 - the ‘plus’ of “ $A+i$ ”
 - the assignment

Counting is “underspecified” (2)

- There can be multiple right answers – if you get ‘2’ and I count ‘4’ then it does not mean you are wrong!
- Note: If I think an answer is ‘4’ then ‘2’ is probably also acceptable – but “2 n” probably will not be.
- It is most important to be able to
 - know what is happening in the underlying process
 - be able to link to C and assembly level notions
 - be able to use this to give a reasonably consistent justification of your answers

Aside: Correctness vs. Efficiency

- Primitive operation counting is relevant to Efficiency – but not (directly) for Correctness
- For correctness, do not care about runtime of the alg.
 - Do care about the time to find a proof if doing an automated search for proofs.
 - The verification seems to be quick in lean: e.g.
`#eval 2035713999 + 350135299`
 - Very quick (<1sec) – not using succ internally (?)
- Note that we did not prove the algorithm correct
 - Would need to do arrays/lists in Lean first
 - Just for thought: And then what?
 - We can do the efficiency of incorrect algorithms 😊

Estimating Running Time: Worst

- We found that getMax uses $6n-2$ primitive operations in the worst case.
- But what about the actual worst-case running time – call it $TW(n)$?
- Not all primitive operations take the same time
 - so cannot directly say the time is proportional to $(6n-2)$
- Instead, suppose the quickest primitive op takes time q , and the slowest time s . Then we have
$$q (6n-2) \leq TW(n) \leq s (6n-2)$$
- $TW(n)$ is sandwiched between two linear functions and so we can say the worst-case running time is linear

Estimating Running Time: Best

- For the best case, the line “max=A[k]” will never run, and then getMax uses $4n$ primitive operations
- For the best-case running time, $TB(n)$ we then have
$$q(4n) \leq TB(n) \leq s(4n)$$
- $TB(n)$ is sandwiched between two linear functions and so we say the best-case running time is linear
- In this case, the best and worst are both linear, but often they might be different:
 - **EXERCISE: Think of examples where the best-case is much better than the worst case**

Remarks

- Do not get too worried with the fine details of counting of primitive operations
- The details of the counting and real timing would depend on:
 - the compiler, and require inspection of the assembly code
 - the CPU architecture, pipelining, cache misses, etc, etc

Growth Rate of Running Time

- Changing the hardware/ software environment
 - Affects runtimes by a constant factor, but
 - Does not alter the growth rate of the running time
- **The linear growth rate of the running times is property of algorithm getMax itself, not just of the actual environment**

Exercise: (exam-style question)

(1)

Suppose we have a routine:

```
int RecDiv(int n) // assume  $n \geq 1$ 
    int m = 0
    while ( $n \geq 2$ )
         $n = n/2$     // integer division, not floating point
        m++
    return m
```

What is the runtime?

I.e. how many primitive operations, $T(n)$, as a function of n ?

And what does it compute?

Exercise:

(2)

Note that the code does $n/2$, and so it will be easier if we assume that the input n is a power of 2, that is $n = 2^k$ for some k .

Now let's start the PrimOps counting:

<code>int RecDiv(int n)</code>	
<code>int m = 0</code>	1
<code>while (n >= 2)</code>	? per pass
<code>n = n/2</code>	? per pass
<code>m++</code>	? per pass
<code>return m</code>	1

What shall we put for the " $n = n/2$ " ?
(Offline? Pause and try it)

Internal Steps:

$$n = n/2$$

?

1. read ***n*** from memory (RAM) and store in a register r1 (very fast piece of memory on the CPU)
2. read **2** from memory and store in a register r2
3. send registers r1 r2 through arithmetic division and store result in a register r3
4. write r3 back to ***n***

CPU steps needed is 4, does not depend on ***n***

Internal Steps: different compiler

$$n = n/2 \quad ?$$

1. read ***n*** from memory (RAM) and store in a register r1 (very fast piece of memory on the CPU)
2. send registers r1 through a right shift of the bits and store result in a register r3
e.g. compute $13/2=6$ by $1101 \rightarrow 110$
3. write r3 back to ***n***

CPU steps needed is 3, different but still does not depend on ***n***

Thought Exercise (offline)

- Based on your knowledge of assembly, and machine architectures try to estimate the number of CPU cycles that might actually be used.
- “Divide” or “shift”? Which is faster?
- **Point: try to eventually build a mental model that is an “internal interpreter” so as to know how a program will run**
- **Such “internal interpreters” are vital for understanding programming (IMHO)**

Exercise:

(3)

Continue with the PrimOps counting:

int RecDiv(int n)	
int m = 0	1
while (n >= 2)	3 per pass
n = n/2	3 per pass
m++	2 per pass
return m	1

Picked some reasonable numbers for the others.
But how many passes?

How many passes through the loop?

Hint: If ever stuck:

- Try simple concrete examples
- Start from “ridiculously simple” and work up to harder examples

(Real mathematicians often work this way; but then hide it when writing up ☹)

- Do a “trace of the program” by hand:

How many passes through loop?

- Focus on the relevant portions

while (n >= 2)

n = n/2

Simplest example?

- Exercise: What is smallest positive integer that is a power of two?
- Answer: 1 as $2^0 = 1$
- (If confused, or if you answered "2", then consider revising your maths about exponents and logarithms)

How many passes through loop? (cont)

while (n >= 2) {n = n/2}

- Case: $n = 1 = 2^0$ passes = 0
- Case: $n = 2 = 2^1$
 - $n=2$, then $n=1$; passes=1
- Case: $n = 4 = 2^2$
 - $n=4$, then $n=2$, then $n=1$; passes = 2
- Case: $n = 8 = 2^3$
 - $n=8, 4, 2$, then $n=1$; passes = 3

How many passes through loop? (cont 2)

- Case: $n = 2^m$
 - $n=2^m, 2^{m-1}, 2^{m-2}, \dots, 2$
 - m passes through loop
- but note, n is the input not m , so want to write answer in terms of n . Use
 - $m = \log_2(n)$
- Result: passes through loop = $\log_2(n)$

Exercise:

(4)

Continue with the PrimOps counting:

int RecDiv(int n)	
int m = 0	1
while (n >= 2)	$3 * (\log_2(n) + 1)$
n = n/2	$3 * \log_2(n)$
m++	$2 * \log_2(n)$
return m	1

Total: $8 \log_2(n) + 5$

(The +1 on line 2 is because the test is done even when it fails)

****Remarks****

- Each pass through the loop the size of n is halved
 - the “ $\log_2(n)$ ” is typical of such “halving on each iteration”
- **This concept also appears in sorting and searching; hence you MUST make sure you fully understand this example. Otherwise, parts of the module will probably be incomprehensible)**

Summary

Goal:

- Build foundations for time-analysis of programs

Skills needed:

- Count primitive operations
- Counting of operations with
 - Loops
 - (Recursion)

Removing details

- According to precisely how we count steps we might get many different answers, e.g. something like
 - $5 \log_2(n) + 2$
 - $9 \log_2(n) + 5$, etc
- Also this counts “steps”
 - the translation to runtime depends on the compiler, hardware, etc
- **Need a way to suppress such details**
 - **but to do it in a well-defined way**

Next Lecture

“Suppressing the details”

A motivation and introduction to big-Oh

Oh... ohh!!

Exercise: (Advanced, Offline)

Suppose we have a routine that takes a sqrt instead of dividing by two:

```
int RecSqrt(int n) // assume n >= 1
    int m = 0
    while (n >= 2)
        n = sqrt(n) // "integer square root" e.g. floor(sqrt(n))
        m++
    return m
```

Repeat the analysis.

For the counting, you can assume the "sqrt" is a primitive operation – though in reality it may be many operations